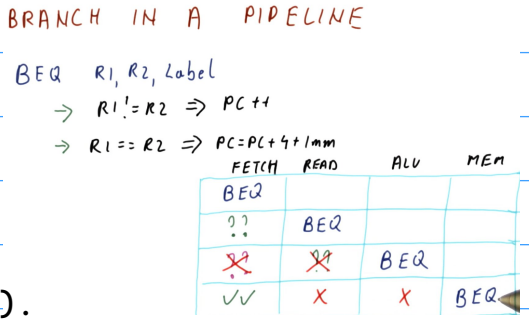
Branch Prediction

Each control hazard waste lots of clock cycles in a pipelined proc. Branches and jumps are relatively common. So to improve performance we need to do something about this.

A branch in a pipeline.
We would need to fetch the next instr. after we fetched a branch, so we have to make prediction even before we know that its a Branch(because not yet decoded).

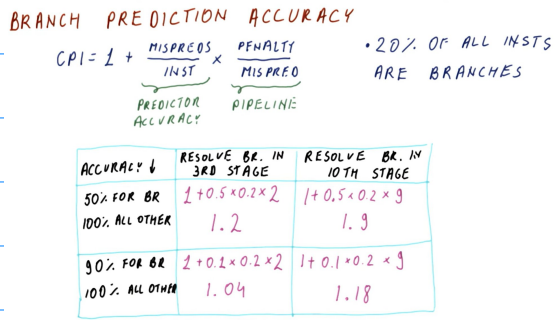


Branch Prediction requirements:

a) We need to guess nxt instr. PC based on knowledge of current instr. PC

b) It should guess: is this a branch, if so taken? (is this a T Branch)

: What is the target addr. if its a T branch



We compare two CPUs where branches get resolved in 3rd and 10th stages .

We get 15% improvement in CPI in shallow pipeline, compared to 61% in deep pipeline with better br. predictor.

The deeper the pipeline, the more dependent it is on a better predictor to give better performance. State of Art Br Pred are >90% accurate but pipelines are so deep that further improvement would help.

BRANCH PREDICTION BENEFIT QUIZ

- 5-STAGE PIPELINE
- BRANCH RESOLVED IN 3RD STAGE
- FETCH NOTHING UNTIL SURE WHAT TO FETCH
- EXECUTE MANY ITERATIONS OF

```
LOOP:
    ADDI  R1,R1,-1
    ADD   R2,R2,R2
    BNE2  R1,LOOP
```

SPEEDUP OF PERFECT PREDICTOR IS _____

BRANCH PREDICTION BENEFIT QUIZ SC

- 5-STAGE PIPELINE
- BRANCH RESOLVED IN 3RD STAGE
- FETCH NOTHING UNTIL SURE WHAT TO FETCH
- EXECUTE MANY ITERATIONS OF

```
LOOP:
    ADDI  R1,R1,-1   2   1
    ADD   R2,R2,R2   2   1
    BNE2  R1,LOOP    3   1
                     7   3
```

SPEEDUP OF PERFE PREDICTOR IS 2.33

PERFORMANCE WITH NOT-TAKEN PREDICTION

REFUSE TO PREDICT

BRANCH: 3 CYCLES

NON-BRANCH: 2 CYCLES

PREDICT NOT-TAKEN

BRANCH: 1 OR 3 CYCLES

NON-BRANCH: 1

ALWAYS WINS

All pipelined processors do some kind of Br Pred, even if its just inc. PC by 1 word. This approach always wins over "not fetching until we are sure what to fetch".

MULTIPLE PREDICTIONS QUIZ




```
BNE R1,R2,LABEL A   T
BNE R1,R3,LABEL B   T
A
B
C
LABEL A:
X
Y
LABEL B:
Z
```

- USING NOT-TAKEN PREDICTOR

HOW MANY CYCLES ARE WASTED ON MISPREDICTIONS UNTIL WE GET TO Y?

Only two cycles are wasted, until we get to Y. We actually make two mispredictions, but second one is shadowded by first mispred.

The second branch would never have been fetched if we predicted first correctly.

However the penality is unaffected.

2nd branch is flushed before it could trigger flush.

PREDICT NOT-TAKEN

- INCREMENT PC
- 20% OF INSTRUCTIONS ARE BRANCHES
- 60% OF BRANCHES ARE TAKEN

CORRECT: 80% + 8%

INCORRECT: 12%

CPI = 1 + 0.12 × PENALTY

If we use "Predict NT" predictor, we would be correct 88% of the time assuming typical real world scenerios. For a 5 stage pipeline, we would get 1.24.

WHY WE NEED BETTER PREDICTION?

	NOT TAKEN 88%	BETTER 99%	SPEEDUP
5-STAGE PIPE (3RD STAGE)	1+0.12×2 = 1.24	1+0.01×2 = 1.02	1.22
14-STAGE (11TH STAGE)	1+0.12×10 = 2.2	1+0.01×10 = 1.1	2
11TH STAGE 4INST/CYCLE	0.25+0.12×10 = 1.45	0.25+0.01×10 = 0.35	4.14!

Say you have a 5 stage pipeline, 14 stage pipeline with branch resolutn in 11 th stage and same with 4 Inst/clk

So CPI is improved by 4x in case of 4IPC processor with better predictor.

PREDICTOR IMPACT QUIZ

- PENTIUM 4 "PRESCOTT"
  - FETCH, ... 29 STAGES..., RESOLVE BRANCHES
  - BRANCH PREDICTION
  - MULTIPLE INSTS/CYCLE
- PROGRAM
  - 20% OF INSTS ARE BRANCHES
  - 1% OF BRANCHES MISPREDICTED
  - CPI = 0.5
- IF 2% OF BRANCHES MISPREDICTED, CPI = _____

We are given the actual CPI, we need to compute the ideal CPI after we remove the branch penality.

Then we compute the new branch penality and add it to ideal CPI.

- 1% OF BRANCHES MISPREDICTED
- CPI = 0.5 = X + 0.2×0.01×30 ⇒ X = 0.5 − 0.06 = 0.44
- IF 2% OF BRANCHES MISPREDICTED, CPI = 0.56

X + 0.2×0.02×30

WHY WE NEED BETTER PREDICTION?

	NOT TAKEN 88%	BETTER 99%	SPEEDUP
5-STAGE PIPE (3RD STAGE)	1+0.12×2 = 1.24	1+0.01×2 = 1.02	1.22
14-STAGE (11TH STAGE)	1+0.12×10 = 2.2	1+0.01×10 = 1.1	2
11TH STAGE 4INST/CYCLE	0.25+0.12×10 = 1.45	0.25+0.01×10 = 0.35	4.14!



Wastage of instr. on mispred. is much higher on higher IPC processor.

BETTER PREDICTION - HOW?

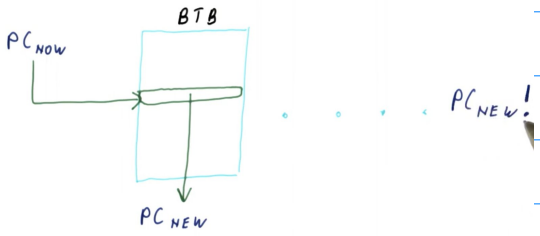
$PC_{NEXT} = f(PC_{NOW})$

- BRANCH ?
- TAKEN ?
- OFFSET ?

$PC_{NEXT} = f(PC_{NOW}, HISTORY[PC_{NOW}])$

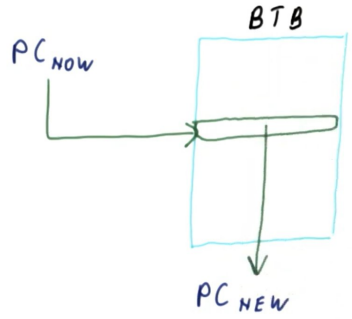
We dont know if the instr. we are going to fetch is a branch, if its taken , the addr of nxt instr. But we know how that instr. behaved last few times it was fetched. So based on this we are going to build our pred. function. The simplest pred. that uses branch history is BTB.

BTB - BRANCH TARGET BUFFER



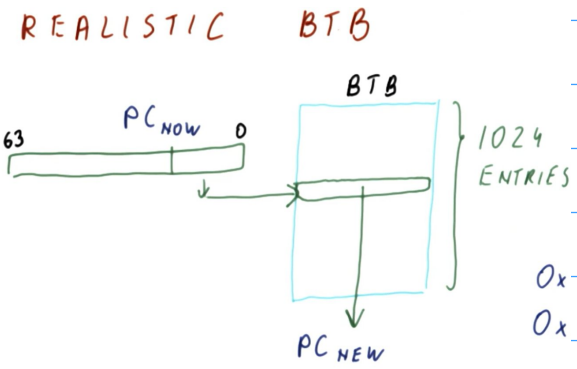
We will index into a table to get prediction, and later in the pipeline, when branch is resolved, it its a mispred, we use same PC_now to index into BTB and change the entry to store correct prediction, so nxt time we get correct prediction assuming it is jumping to the same location.

BTB - HOW BIG!



- 1 CYCLE LATENCY
- SMALL!!
- 8-BYTE ENTRY
- 1 ENTRY / PC

=> LARGE!



In reality, We don't actually need 1entry/ PC for all the program, we just need enough entries for instr. that we are going to execute soon. Say we have 100 instr. loop, we just need entries just above 100 in the BTB.

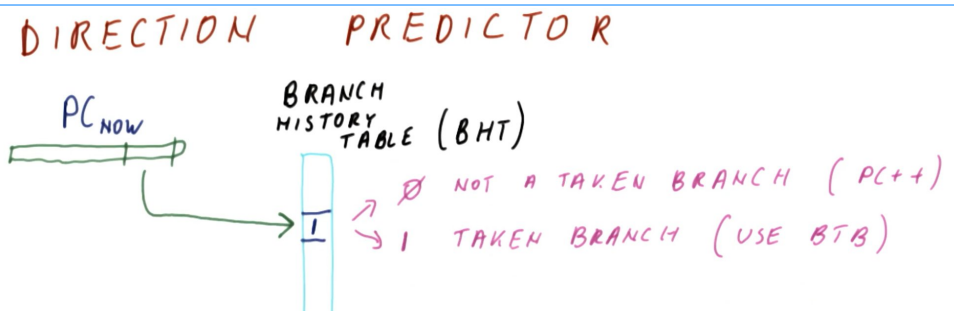
Say we have done timing exp and found that we can have 1024 entries, that can be accessed in a single cycle. We use LSB bits of PC addr. to index into BTB. this is simplest mapping function that fits in timing budget, i.e mapping function + BTB read can fit in single cycle.

BTB QUIZ

- BTB HAS 1024 ENTRIES (0..1023)
- FIXED-SIZE 4-BYTE INSTRUCTIONS, WORD-ALIGNED
- 32-BIT ADDRESS

WHICH BTB ENTRY IS USED FOR PC = 0x0000AB0C

10_1100_0011 = 0x2c3



After we resolved the branch and found to be taken B, then we index into BHT and set that bit, and write the target addr. into BTB. else we reset BHT bit and don't update the BTB. BHT can be quite large as only one bit, While still reserving small BTB , that's just for Branches.

BTB & BHT QUIZ

BHT: 16 ENTRIES, PERFECT
BTB: 4 ENTRIES, PERF.

0xC000		MOV R2, 100	
0xC004		MOV R1, 0	
0xC008	Loop:	BEQ R1, R2, Done	
0xC00C		ADD R4, R3, R1	
0xC010		LW R4, 0(R4)	
0xC014		ADD R5, R5, R4	
0xC018		ADD R1, R1, 1	
0xC01C		B Loop	
0xC020	Done:		

HOW MANY TIMES DO WE ACCESS THE BHT FOR EACH INSTRUCTION ?



Suppose we have BTB and BHT as perfect predictors.

BHT is accessed every time we fetch a instr. to see it its a taken Branch.

0xC000		MOV R2, 100	1
0xC004		MOV R1, 0	1
0xC008	Loop:	BEQ R1, R2, Done	101
0xC00C		ADD R4, R3, R1	100
0xC010		LW R4, 0(R4)	100
0xC014		ADD R5, R5, R4	100
0xC018		ADD R1, R1, 1	100
0xC01C		B Loop	100
0xC020	Done:		

Same Program, and setup as above (16 entry bht , perfect & 4 entry btb)

Q. is which BHT entry do we access for each instr. (for all 8 instr.)

1100 0000 0000 0100

BT

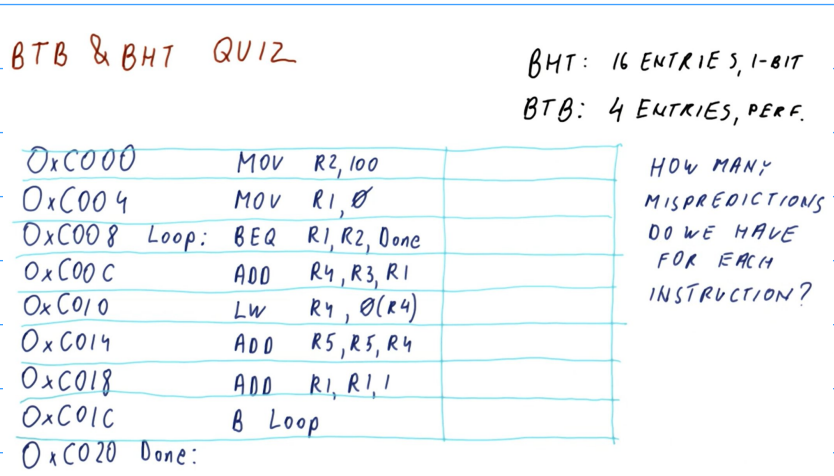
0xC000		MOV R2, 100	0
0xC004		MOV R1, 0	1
0xC008	Loop:	BEQ R1, R2, Done	2
0xC00C		ADD R4, R3, R1	3
0xC010		LW R4, 0(R4)	4
0xC014		ADD R5, R5, R4	5
0xC018		ADD R1, R1, 1	6
0xC01C		B Loop	7
0xC020	Done:		

Same program, and setup as above, Q. is how many times do we need to access BTB for each instruction.

A) BTB will only be accessed when the BHT table tells the instruction is a taken branch. So the "B loop" instr. accesses BTB 100 times, and "BEQ" instr. accesses only once because its taken branch only once, and we are assuming BHT to predict perfectly.`

Q. which btb entry is used for each instr.

A. We use btb only for beq and B instn.s, so the entries are 2 and 3, corresponding to 10 and 11 in [3] and [2] index values of intr. addr.



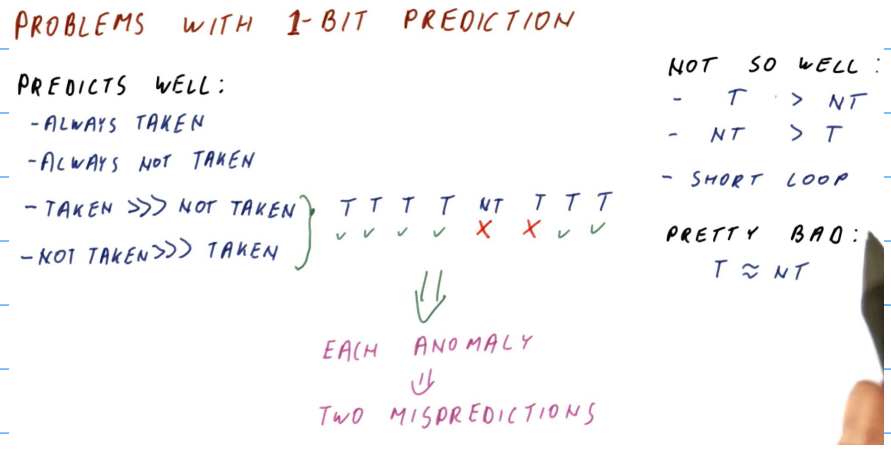
BHT is a 1 bit predictor, which predicts NT , default

1 mispred. for B which is first time its exe.

1 mispred for beq, which is last time its exe.

As it can be seen here, 1 bit predictor works really well, here we just had two mispred. for 600 instr. , so for loops with many iter, it has good accuracy. Later we will see for if-else and less iter its not so good.

Problems with 1- bit predictor



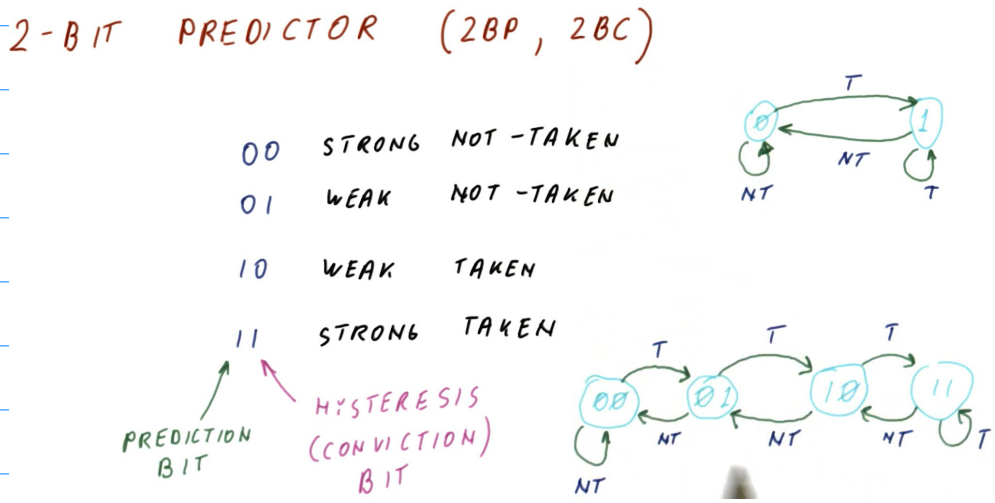
1b predictor is not so well, in case of T and NT branches are comparable in number or almost equal. They are also not so well on short loops inside large loop, so for short loop there will be two mispreds one during enter loop and one during exit loop.

To compensate for "two mispreds per anamoly" behaviour in 1bp, we have 2 bit predictor often called 2bp or 2bc (2Bit Counter). The prediction bit functions just like one bit predictor and conviction bit tells us how sure are we about that prediction.

Single occurance of non-dominant branch dir, will not change the predictn

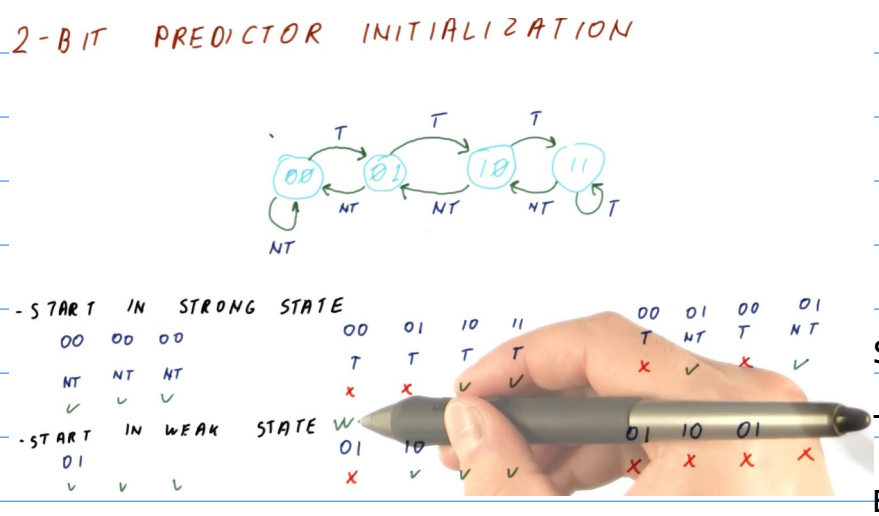
A single anamoly will cost us 1 mispred, and change in behaviour will cost us two mispredictions.

It is called a 2bc, because it counts up and saturates at 3 on T branches and counts down and saturates to 0 if branches are NT.



2 BP predictor initialization

A 2 BP should initially start in strong conviction state or weak predtn st

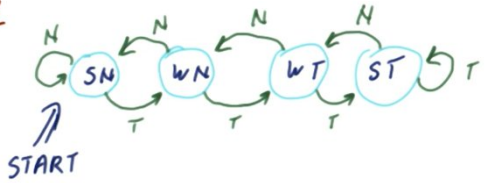


Starting in any initial state doesn't offer any adv. 00 is often choosen cause easier to init.

Some prefer weak NT , becz T are more common than NT. But don't effect accuracy.

2BP QUIZ

- 2BP



- IS THERE A SEQUENCE OF BRANCH OUTCOMES THAT RESULTS IN NEVER PREDICTING CORRECTLY?

YES? SEQUENCE

Even if we start in a SNT state, there is a way to move us into always incorrect prediction. so the pattern is T,T,NT,T,NT,T ...
Every predictor will have a sequence that will generate its worst possible outcome, which produces 100% mispred

its just the matter of how likely is that seq.

1BP → 2BP ☺ 3BP? 4BP?

- BAD: COST ↑
- GOOD: WHEN "ANOMALOUS" OUTCOMES COME IN STREAKS
  - HOW OFTEN DOES THIS HAPPEN?

STAY WITH 2BP, MAYBE 3BP ...

HOW DO WE FURTHER IMPROVE PREDICTION?

N T N T N T N T ...

The idea of using more bit predictors is not so useful because the possibility of having anamalous outcomes in streaks is less. So we may atmost go for 3bp.

So how do we further improve prediction.

t nt t nt is perfectly predictable , but not possible with counter based predictors, so we will focus on this predictors.

HISTORY-BASED PREDICTORS

N T N T N T . . .

N N T N N T . . .

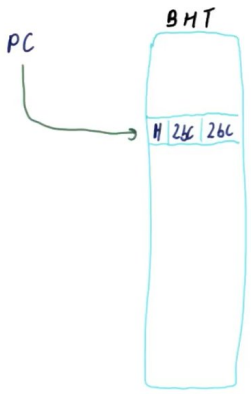
} 100 % PREDICTABLE BUT NOT WITH SIMPLE N-BIT COUNTER

LEARN THE PATTERN!

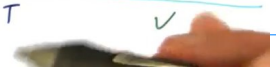
	N	T	N	T	N	T . . .
HISTORY		N	T			
PREDICTION		T	N			
HISTORY	N	N	T	N	N	T
			NN	NT	TN	
			T	N	N	



1-BIT HISTORY WITH 2-BIT COUNTERS



STATE	PRED	OUTCOME	CORRECT ?
(0, SN, SN)	N	T	✗
(1, WN, SN)	N	N	✓
(0, WN, SN)	N	T	✗
(1, WT, SN)	N	N	✓
(0, WT, SN)	T	T	✓
(1, ST, SN)	N	N	✓
(0, ST, SN)	T	T	✓



1-BIT HISTORY QUIZ

- 1 BIT HISTORY (START 0)
- 2BC/HISTORY (START SN)
- (NNT)*

AFTER 100 REPETITIONS, THE # OF MISPREDICTIONS IS _____

1-BIT HISTORY QUIZ

- 1 BIT HISTORY (START 0)
- 2BC/HISTORY (START SN)
- (NNT)*

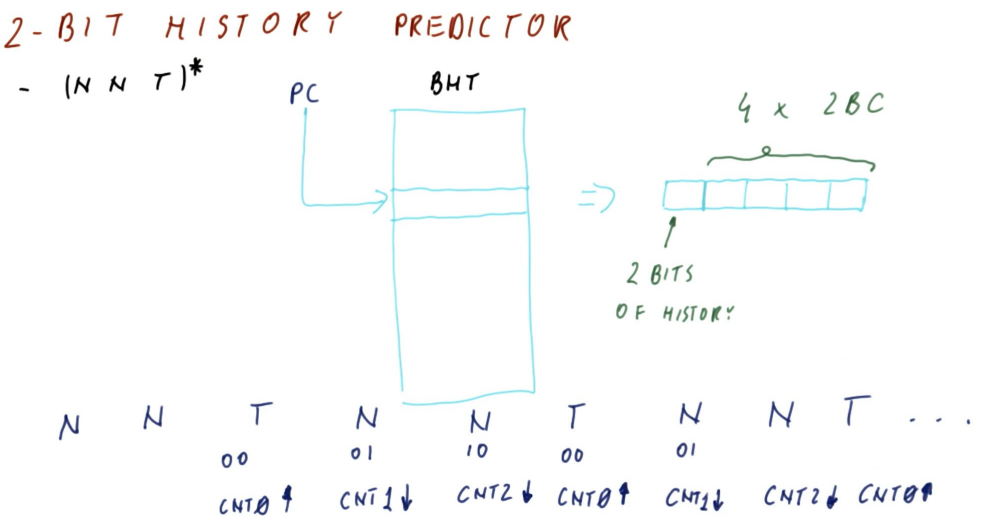
AFTER 100 REPETITIONS, THE # OF MISPREDICTIONS IS 100

SOLUTION

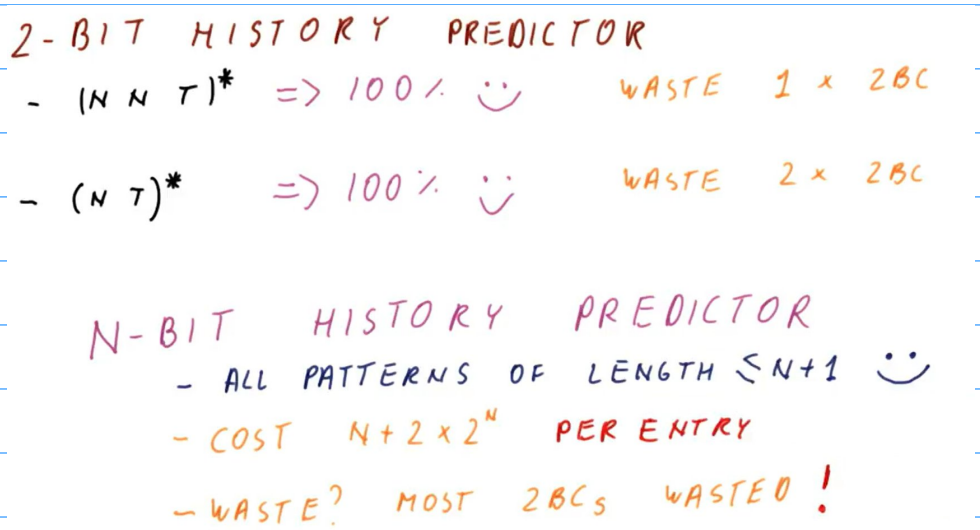
STATE	PRED	OUTCOME	?
0, SN, SN	N	N	✓
0, SN, SN	N	N	✓
0, SN, SN	N	T	✗
1, WN, SN	N	N	✓
0, WN, SN	N	N	✓
0, SN, SN	N	T	✗
		N	
		N	
		T	
		⋮	



As we can see above ex., the 1bit history predictor is not so good for this kind of pattern

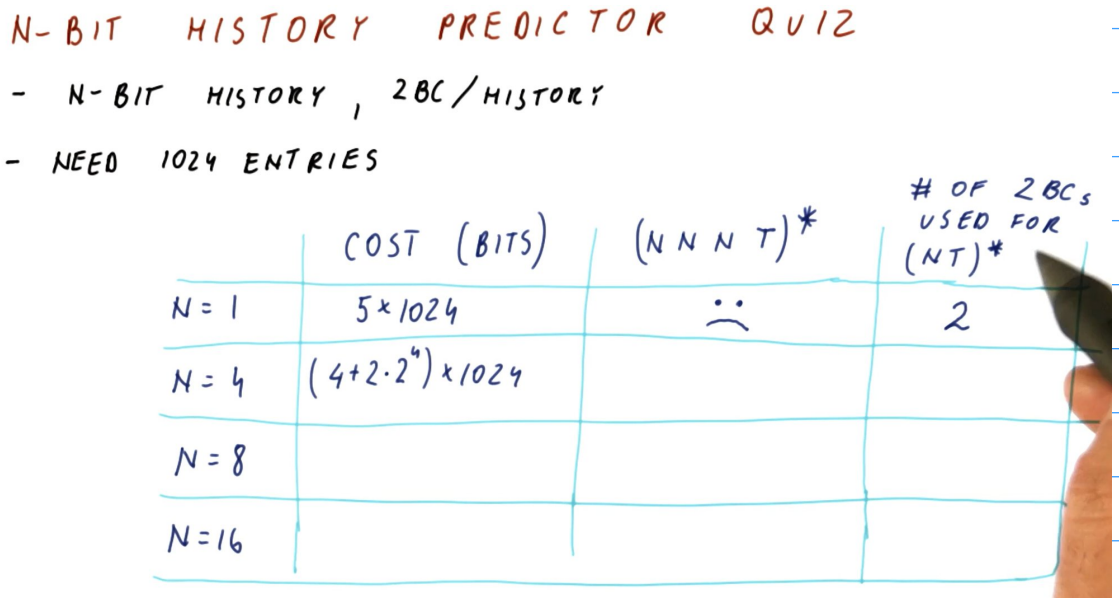


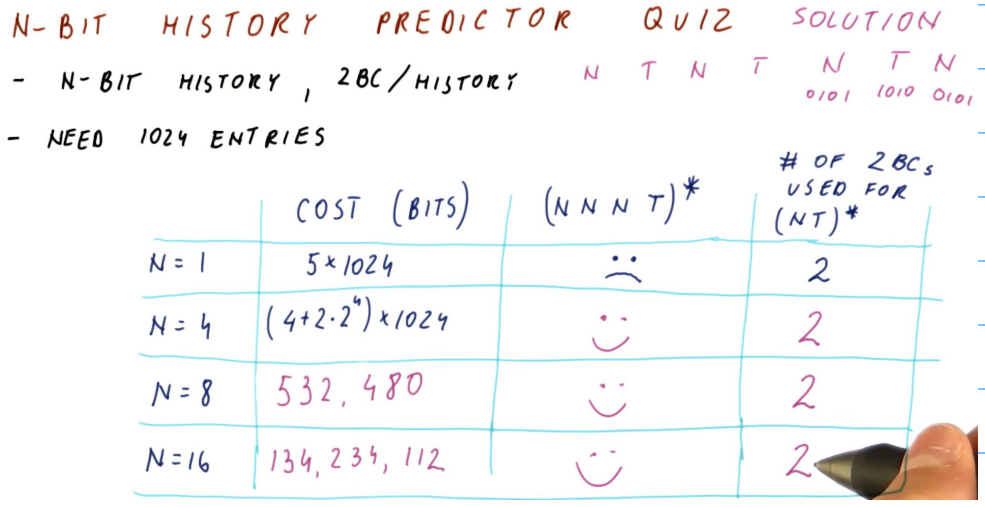
So this 2 bit history needs 10 bits per branch. Very soon each counter starts predicting the correct branch. In this case, cnt0 will saturate to ST, and cnt1 and cnt2 will saturate to SNT. cnt3 will be unused because we never see history of TT.



After initial warmup period, these predictors predict with 100% accuracy. Most of the 2BCs are wasted, for ex. we have 10 bit history, we would have 1024 2BCs , We can only predict patterns of upto 11 in length, and 11 length pattern will have atmost 11 histories. So here out of 1024 , we are using only 11. So longer histories , are expensive with # of 2BCs and utilization factor also drops.

Say we have a 8 iter loop, which is 7N then followed by T, say this loop is executed inside another big loop, then we would need 7 or 8 bit history So longer histories are common and are needed, but we need to deal with wastage and huge costs somehow.





For N=16, it is not a realistic predictor for the size. and for N=8, we probably might not want to spend so much hardware for just Branch pred. Any pred with N ≥ 3, will predict NNNT* pattern correctly. For N=4, we have 0101 as history for NT prediction and 1010 for T prediction, in a N T * pattern. In case of N=16, we will be using only 2 out of 65536 2BCs in each entry.

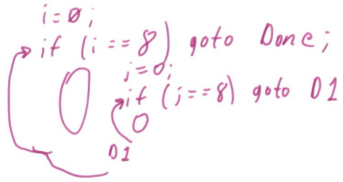
HISTORY PREDICTOR QUIZ

```
for (int i=0; i!=8; i++)
    for (int j=0; j!=8, j++)
        do something;
```

AT LEAST _____ ENTRIES
EACH ENTRY SHOULD HAVE
AT LEAST _____ -BIT HISTORY
(=> _____ 2-BIT COUNTERS)

HISTORY PREDICTOR QUIZ SOLUTION

```
for (int i=0; i!=8; i++)
    for (int j=0; j!=8, j++)
        do something;
```



AT LEAST 4 ENTRIES
EACH ENTRY SHOULD HAVE
AT LEAST 8 -BIT HISTORY
(=> 256 2-BIT COUNTERS)

(N N N N N N N N T)*

We need to have entry for each instr. but atleast we need to have 1 entry per each branch, so there are 4 branches. We don't need history for loop back branches because they are always taken (even a single 2BC will do well on these) . And for loops we have 8#of N and followed by T, so we have 9 long history, so 8 bit history would work. 2^8 = 256 2BCs needed. For both the branches, the pattern is (NNNN NNNN T)*, and how many patterns we have ??, we have just 9 patterns. So we use only 9 2BCs.

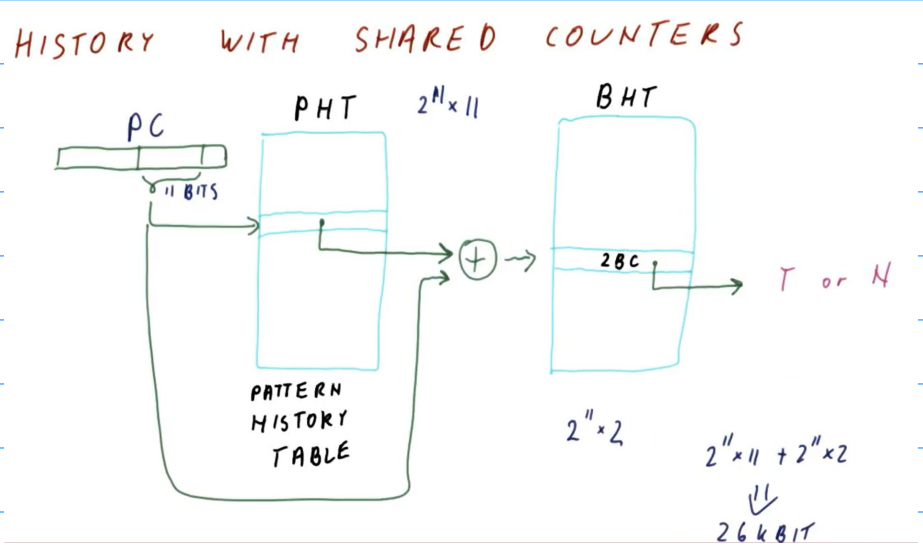
HISTORY WITH SHARED COUNTERS

N-BIT HISTORY => 2^N COUNTERS PER ENTRY

ONLY ≈ N COUNTERS USED

IDEA: SHARE 2BCs BETWEEN ENTRIES

The idea is to share the 2BCs between entries , instead of dedicating 2^N 2BCs for each entry. But it is possible that different entries (with diff PCs and diff histories end up using same 2BC. There is possiblity of conflict but if we have enough of 2BCs the chances are less, because each entry doesn't need too many of them.



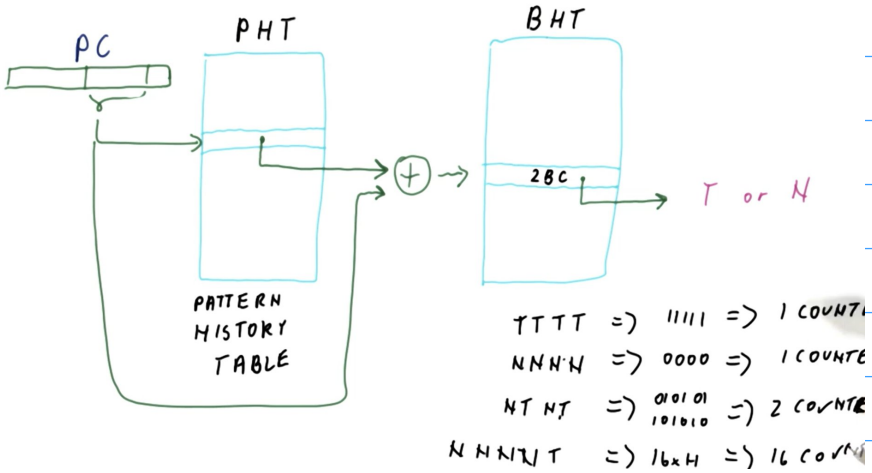
Here we are assuming 11 bit history, and we are using 11b of PC to index into PHT.

PHT simply keeps the history bits alone for that branch.

When the branch outcome is known, we use same history (from PHT) and PC combination to index into 2BC and inc. or dec. it based on the current outcome , also we will shift the branch outcome into PHT, to make this history ready for next prediction on this branch.

PC tells us which branch we are at. PHT hopefully has one history per branch, the f(xor here) of PC and PHT gives us a 2BC.

HISTORY WITH SHARED COUNTERS

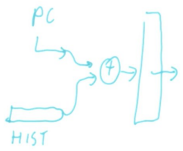


PSHARE

PRIVATE HISTORY
SHARED COUNTERS
GOOD FOR
EVEN-ODD
8-ITERATION LOOP

GSHARE

GLOBAL HISTORY
SHARED COUNTERS
GOOD FOR
CORRELATED BRANCHES




```
if(shape == square)
  ~~
~~~
if (shape != square)
  ~~
```

PSHARE VS GSHARE QUIZ

```
for (int i= 1000; i!=0; i--)
  if(i%2)
    nt= i;
```

GOOD ACCURACY ON ALL BRANCHES => HISTORY SHOULD BE?

FOR PSHARE _____

FOR GSHARE _____

```
LOOP:
    BEQ   R1, Zero, Exit
    AND   R2, R1, 1
    BEQ   R2, Zero, Even
    ADD   R3, R3, R1
Even:
    ADD   R1, R1, -1
    B     Loop
Exit:
```

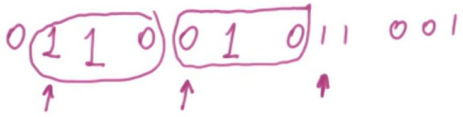
The BEQ_1 and B instr. do not need histories, B is always taken, and BEQ_1 is always NT is fine (1 mispred out of 1001). BEQ_2 is even_odd branch. So for PShare, hist len 1 works fine.

FOR PSHARE 1

FOR GSHARE 3

B Loop

Exit:



BEQ_1 is 0, BEQ_2 is taken for the first time(as zero is even), then B is taken, so 011, for nxt iter of loop it is 001. And this is repeating So to predict BEQ_2(marked with arrow), we would need atleast last three. And in this case only two possible histories are possible 110 and 010.

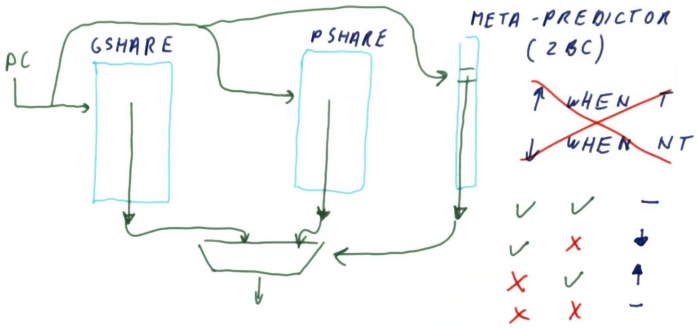
Pshare is better with self similar? branches even Pshare has shorter history, can predict well. And Gshare can do well with correlated branches . So modern processors include both.

TOURNAMENT PREDICTOR

TWO PREDICTORS

ONE BETTER FOR BRANCHES X, Y, Z

OTHER BETTER FOR BRANCHES A, B, C



HIERARCHICAL PREDICTORS

TOURNAMENT

- 2 GOOD PREDICTORS
- UPDATE BOTH ON EACH DECISION

HIERARCHICAL

- 1 GOOD, 1 OK PREDICTOR
- UPDATE OK-PRED ON EACH DECISION, UPDATE GOOD-PRED ONLY IF OK-PRED NOT GOOD

Hierarchichal predictor example

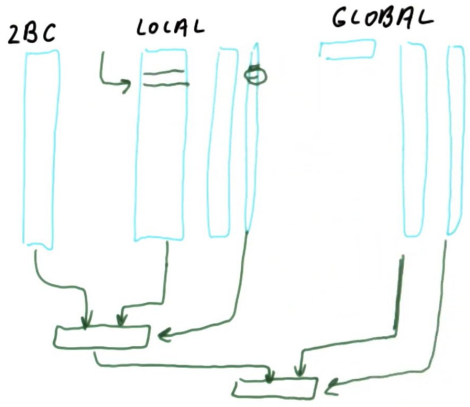
The proc. has large array of 2BCs. A local history table, and 2BCs.A global history, with global histories and 2BCs. The global predictor has a tag array (which can have some upper bits of the PC) which says if that branch is gonna predicted by global predictor. If global pred. doesn't have a matching tag, we use prediction from local predictor. Local predictor also has a tag array, if that branch is not in local pred tag array , we just use the result of the 2BC.

When we update the predictor, we update the 2BC, if we have the branch in local pred. we update the local pred. , if we have the branch in global pred. we update the global pred. . If we update the 2BC, and the branch is not in local pred., and 2BC is mispredicting it, we insert that branch in local pred., now the branch will be present here next time. Similarly, if local pred. not giving good pred. we insert the branch in global pred.

How do we say if the branch is in local predictor. We insert some bits of addr. into tag bits.Its usually upper few bits of PC.So diff branches that index into this Local pred's history line, only one of them will be pred by this predictor based on the tag bits, rest of them say they are not found here.

HIERARCHICAL PREDICTOR EXAMPLE

- PENTIUM M PREDICTORS
  - CHEAP (2BC)
  - LOCAL HISTORY
  - GLOBAL HISTORY



MULTI-PREDICTOR QUIZ

- 2BP WORKS FINE FOR 95% OF INSTRUCTIONS
- PSHARE WORKS FINE FOR THE SAME 95%, PLUS 2% MORE
- GSHARE WORKS FINE FOR THE SAME 95%, PLUS THE OTHER 3%

THE OVERALL PREDICTOR IS A [ ] PREDICTOR THAT CHOOSES BETWEEN A [ ] PREDICTOR AND A [ ] PREDICTOR, WHICH ITSELF CHOOSES BETWEEN [ ] AND [ ].

A: 2BP
B: PSHARE
C: GSHARE
D: TOURNAMENT
E: HIERARCHICAL
F: RAS

soLeadbc

For conditional branches, we need to predict direction, for which we have seen diff kinds of predictors. and we need to predict target addr. if is T for that a simple BTB will do just fine.
Another type of branches are unconditional jumps, func. calls etc. their dir is always taken, even the simplest predictor type(1b bht) of dir predictors will do well here and for target, we have BTB.

Function returns are type of branches that is always taken, so dir pred is easy, but the return addr. target is variable because we call this func from diff locations in the program. The BTB in this case will not do too well. So the question is how to predict return addresses accurately.

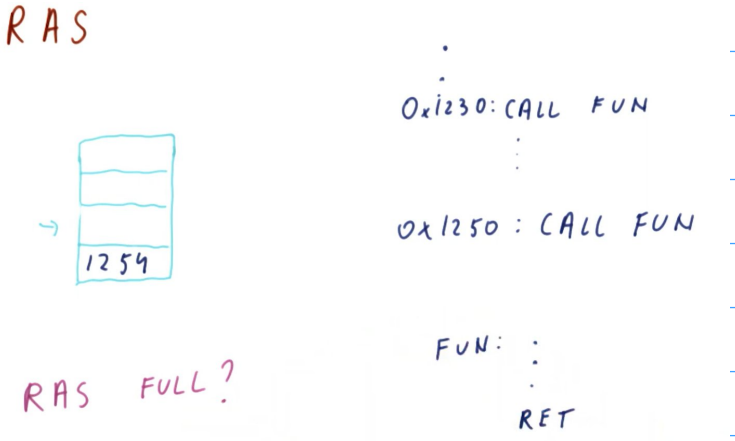
RETURN ADDRESS STACK (RAS)



The solution is we use separate predictor called RAS to predict return addresses.The way this works is, when a func is called, we push the PC onto stack and inc. the pointer. And when we encounter return we dec the pointer and read the return addr. (we don't need to delete that because if pointer is here means that location is empty)

Why is this a predictor, why not use actual stack of the program??

This predictor needs to be very close on chip where rest of branch pred. is happening and should be very small. SO we can get pred like in 1 cycle

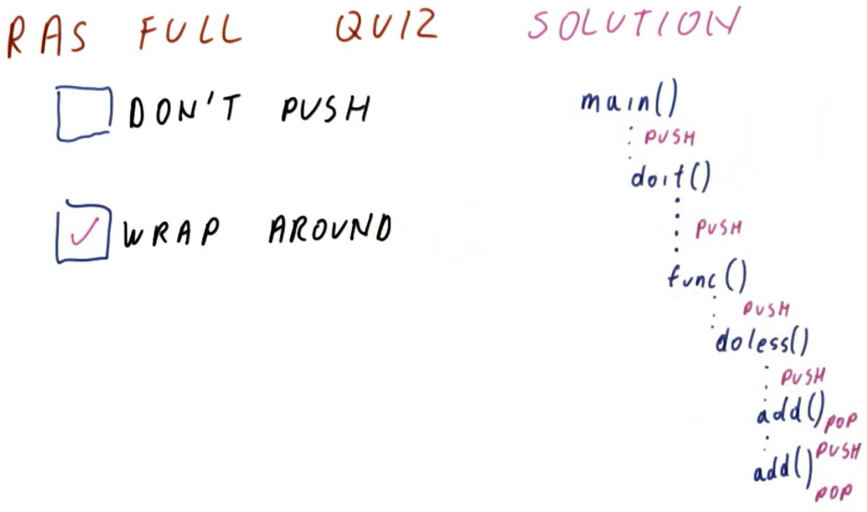


Which is better when RAS full

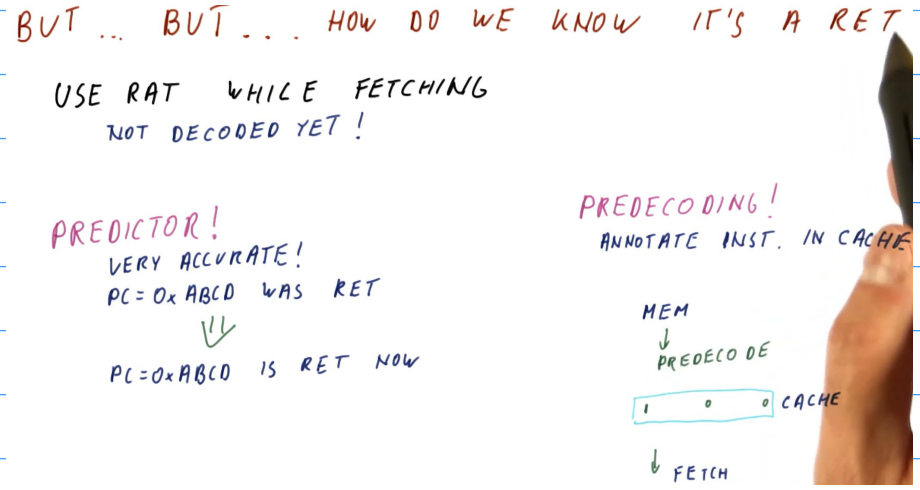
* Dont Push
* Wrap Around

It turns out that wrapping around is much better. Say we have only two entries, the first one is filled when main calls doit(), and second entry will be consumed when we call func() from doit(). So inside doit(), we might have very small functions that willl be called many many times. IF we don't push when RAS is full we will not be correctly predicting these many functions, just the last return of the big/long functions will be predicted correctly. So we are using these two entries here just to save two mispreds at the final few returns.

In the wrap around approach, we will be correctly predicting calls to small functions and returns trading off correct predictions for last two function returns. Hence we are more effectively utilizing the RAS entries



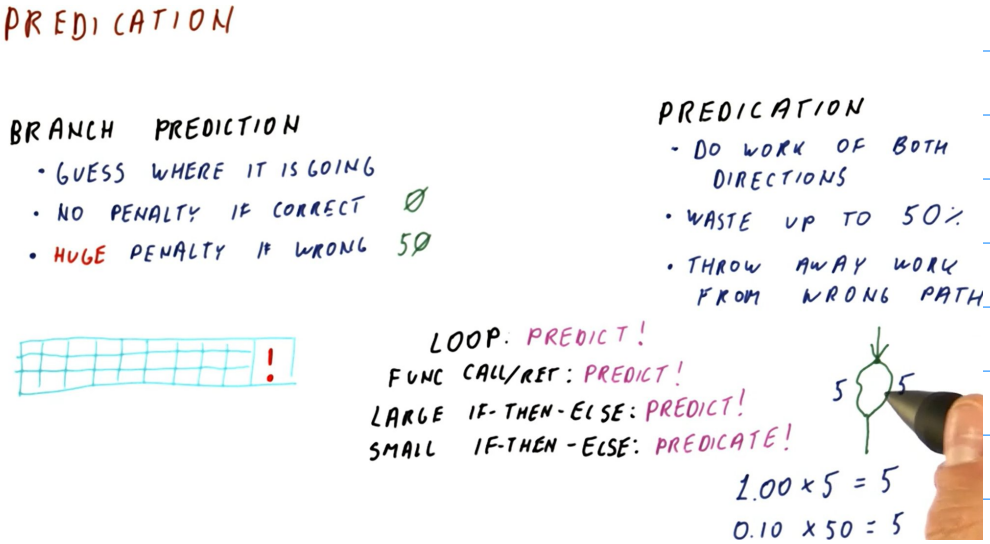
Predecoding can be used to tell is it a return inst. / is it a branch inst inst. len if instns are variable length.



So we are done with prediction lesson. Some branches are very difficult to predict even when we have sophisticated predictors.

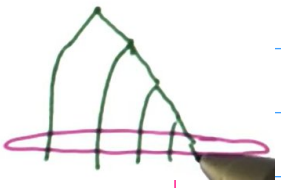
So our next chapter is about helping the compiler completely eliminate these branches from the program so we dont have to predict them

Predication is another way we try to deal with control hazards.



Let's compare prediction vs predication on diff types of branches that we might want to predict.

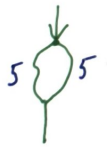
Loops: We do half instrns. for loop taken, and half instn.s from after the loop. Very soon after few iterations we will be doing a lot of work that is after the loop, and the actual useful work will be very less.



After a while out of all the works we are doing , a very small frac. ends up being the "staying in the loop" work. And a lot of it is various variants of work that is after the loop.

Func calls and Returns: Predication doesn't make sense, because we always go there, so direction of not going there does not exist.

Large if-then-else: We would be executing both the if and else parts, So if we have like 100 instns under if and under else, then we would be wasting 100 instns anyway, atlease with prediction we waste only 50 (in this case, also typically) that too when we mispredict. So better Predict



Small if-else: Say we have 5 instructions under if and under else, so if mispred wates 50 instns then having 90% or grater branch predictor accuracy will make us use prediction , else predication would work

$1.00 \times 5 = 5$

$0.10 \times 50 = 5$

LOOP. PREDICT!
FUNC CALL/RET: PREDICT!
LARGE IF-THEN-ELSE: PREDICT!
SMALL IF-THEN-ELSE: PREDICATE!

If the "if-else" is even smaller than 5-5 instructions, then it biases things towards in favour of predication.

Before we dive into how predication works in hardware, lets first see into a technique called if-conversion, through which compiler creates a code that will be executed along both paths.

IF-CONVERSION

```
if (cond) {
    x = arr[i];
    y = y+1;
} else {
    x = arr[j];
    y = y-1;
}
```

IF CONVERSION ⇒

```
x1 = arr[i];
x2 = arr[j];
y1 = y + 1;
y2 = y - 1;
x = cond ? x1 : x2;
y = cond ? y1 : y2;
```

```
BEQ ...,
MOV x, x2
B Done
MOV x, x1
```

If we were to again use this conditional execution as shown above, we haven't done much, we basically just converted one kind of branch into ano And we have this new type of branch twice, and we need some sort of correlating predictor like GShare, and we will likely have two mispreds, instead of one before conversion. If this is all we can do , then we dont do IF-conversion.

For IF conversion to work, we would need some kind of MOV instn. that does MOV X,X1,cond.

CONDITIONAL MOVE

MIPS

```
MOVZ Rd, Rs, Rt   ⇒  if(Rt == 0) Rd = Rs;
MOVN Rd, Rs, Rt   ⇒  if(Rt != 0) Rd = Rs;
```

```
x = cond ? x1 : x2;
R3 = cond;
R1 = ..x1...
R2 = ...x2...
MOVN X, R1, R3
MOVZ X, R2, R3
```

x 86

CMOVZ, CMOVNZ, CMOVGT, ... ⇒ if(cc) Dst = src;

MOVZ/MOVN QUIZ

```
BEQZ R1, Else
ADDI R2, R2, 1
B End
Else:
ADDI R3, R3, 1
End:
```

IF CONVERSION ⇒

```
ADDI R4, R2, 1
ADDI R5, R3, 1
```

Sol is :

```
MOVZ R3, R5, R1
MOVN R2, R4, R1
```

MOVZ/MOVN PERFORMANCE

80%, 40-INST PENALTY ON MISPRED

```
BEQZ  R1, Else
ADDI  R2, R2, 1
B     End
Else:
ADDI  R3, R3, 1
End:
```

IF CONVERSION ⇒

```
ADDI  R4, R2, 1
ADDI  R5, R3, 1
MOVN  R2, R4, R1
MOVZ  R3, R5, R1
```

4

2.5 + 0.2 × 40

⇓

10.5

Say instn B is perfectly predicted always. And BEQ is 80% with 40 instn penality.

Code on left costs us 3 instns on eq.0 and 2 instns on neq. 0

So on avg 2.5, considering,

it is a very difficult to predict branch, so its not biased.

So before if conv. we have on avg. 10.5 instns to execute , whereas after if conversion, we just have 4.

MOVZ/MOVN PERFORMANCE QUIZ

50% TAKEN

```
BEQZ  R1, Else
ADDI  R2, R2, 1
B     End
Else:
ADDI  R3, R3, 1
End:
```

IF CONVERSION ⇒

```
ADDI  R4, R2, 1
ADDI  R5, R3, 1
MOVN  R2, R4, R1
MOVZ  R3, R5, R1
```

IF-CONVERSION BETTER WHEN PREDICTION ACCURACY IS BELOW _____

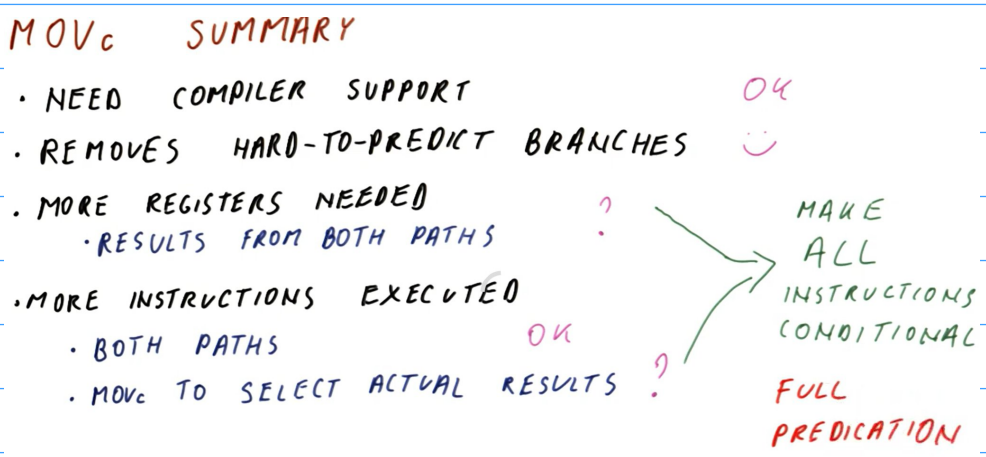
ASSUMING 30-INSTRUCTION MISPRED PENALTY

95%

Here we use 50% to compute avg instns. on perfect prediction. i.e 2.5 here.

$$2.5 + X \cdot 30 = 4$$

$$X = \frac{1.5}{30} = 0.05 \ (5\%)$$



We need compiler support, so this code is not a fully backwards compatible approach, in sense that if we add movc instn. support in hardware, we are allowing compiler to take advantage of that, but old code will not still use these conditional instns.

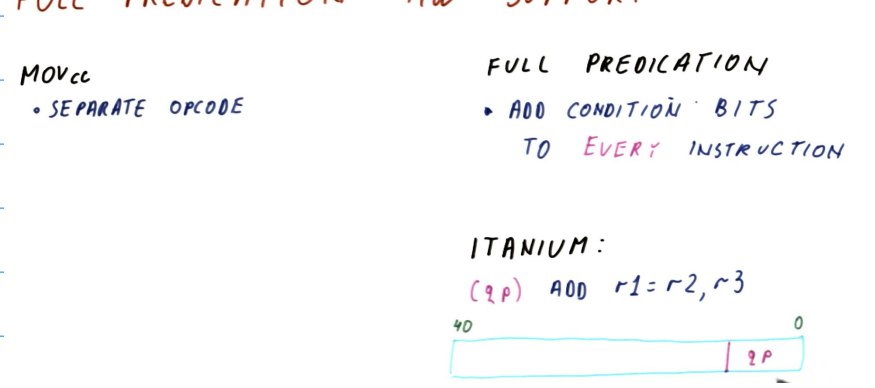
Through profiling (there can be other methods),

we determine that particular a branch is hard to predict

we can perform if conversion on them and remove them.

Needing compiler support and executing both paths are the nature of predication and we can't do anything about that. But can we do anything about Needing more registers or additional instructions in the ISA?

The solution is Full predication . i.e every instn. computes its results and writes to / doesn't write to destination register based on a condition is true or not. Full predication requires extensive support in the ISA.



In the itanium, we have predicate set instn. that does the condition check. It compares eq.0 on R1 and sets P1. P1 and P2 predicates are always set complementary by the instn. We will predicate ADD instn. with P2. So is the case with second ADD instn. So as we can see, once we do predicate set instn. , we just predicate all of the instns with one of the conditions . So we can use same registers as before, we dont need R4 and R5. And we dont have MovC instns. So the overhead is same as we had with branch instns i.e MP.EQZ instn in this case instead of BEQ. The overhead of doing instns from both paths is there, but there is no overhead of selecting the results.

FULL PREDICATION EXAMPLE

```
BEQZ  R1, Else
ADDI  R2, R2, 1
B  End
Else:
ADDI  R3, R3, 1
End:
```

⇒ IF CONVERT

```
MP.EQZ  P1, P2, R1
(P2)  ADDI  R2, R2, 1
(P1)  ADDI  R3, R3, 1
```



FULL PREDICATION QUIZ

```
BEQZ  R2, ELSE
ADD   R1, R1, 1
B     DONE
ELSE:
ADD   R1, R1, -1
DONE:
```

⇒ IF-CONVERSION

```
                R2==0   R2!=0
MP.EQZ  P1, P2, R2
(  )  ADD  R1, R1, 1
(  )  ADD  R1, R1, -1
```

Sol :

P2

P1

- ASSUMING
  - CPI = 0.5 WITHOUT MISPREDICTIONS
  - MISPRED PENALTY 10 CYCLES

⇒ DO IF-CONVERSION IF BEQZ PREDICTION IS < _____ % CORRECT

After IF conversion, we have 3 instns, with CPI of 0.5, we always take 1.5 cycles.

In original code, if NT, 2 inst if branch is T, we do 3 instns

Lets say on avg, we do 2.5 instns. so 1.25 cycles ideal, without mispred

1.25 x 10*mispred% = 1.5 , we get mispred % as 2.5%, so we need ≥ 97.5% prediction accuracy to avoid predication.

We convert difficult to predict branches into code that is longer but no branches. We also seen spl. instns. that we need for IF conversion to work

Now we are upto speed on how ctrl hazards are handled, We need to see how to handle data hazards, while keeping good instn. flow in the pipeline